MODULE 05

Thread Programming in Python

Threads, locks, coordination, executors, processes and asyncio

```
>>> import threading, asyncio
```

AIN3005 · Advanced Python Programming

Binnur Kurt, PhD · Bahçeşehir University · AI Engineering



Learning Objectives

By the end of this module, you will be able to:

1. Distinguish concurrency from parallelism and threads from processes
2. Explain the GIL and predict when threads speed up CPU- or I/O-bound work
3. Create, start, join, name and daemonise threads with `threading`
4. Detect race conditions and deadlocks; prevent them with locks and lock ordering
5. Coordinate threads with semaphores, events, conditions, barriers and queues
6. Run tasks with `ThreadPoolExecutor`, `ProcessPoolExecutor` and futures
7. Write `asyncio` programs with tasks, `TaskGroup` and timeouts
8. Choose a suitable concurrency model for a given workload

MODULE 05 · THREAD PROGRAMMING

Module Roadmap

01 **Concurrency Concepts**
Concurrency vs parallelism, processes vs threads and the GIL

02 **Working with Threads**
Creating, starting, joining and naming threads with the threading module

03 **Race Conditions and Locks**
Lost updates, mutual exclusion, thread-safe classes and deadlock

04 **Coordination Primitives**
Semaphores, events, conditions, barriers and queues

05 **Executors and Processes**
Thread pools, futures, multiprocessing and choosing a model

06 **Asynchronous Programming with asyncio**
Coroutines, the event loop, tasks, TaskGroup and timeouts

01

Concurrency Concepts

Concurrency vs parallelism, processes vs threads and the GIL

- Concurrency vs parallelism
- Processes and threads
- CPU-bound vs I/O-bound work
- The Global Interpreter Lock and its future

Why Write Concurrent Programs?

Responsiveness
- The UI keeps reacting while work runs
- e.g. an editor saving in the background

Throughput While Waiting
- Start the next request while one waits
- Network, disk and databases are slow

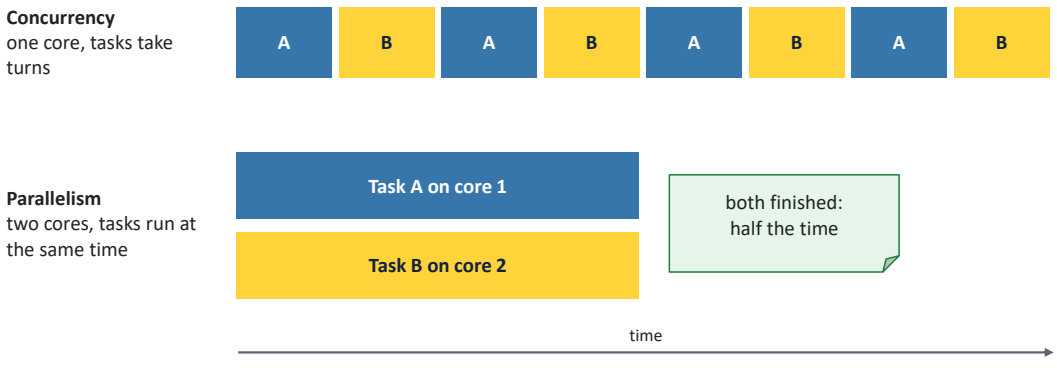
Multi-Core Speed-Up
- Split heavy computation across cores
- Laptops have 8–16 cores

Natural Structure
- A server serves each client independently
- Producers and consumers run side by side

The price

Concurrency has a price: shared data, non-deterministic ordering and harder testing. Use it where waiting or heavy computation dominates.

Concurrency vs Parallelism

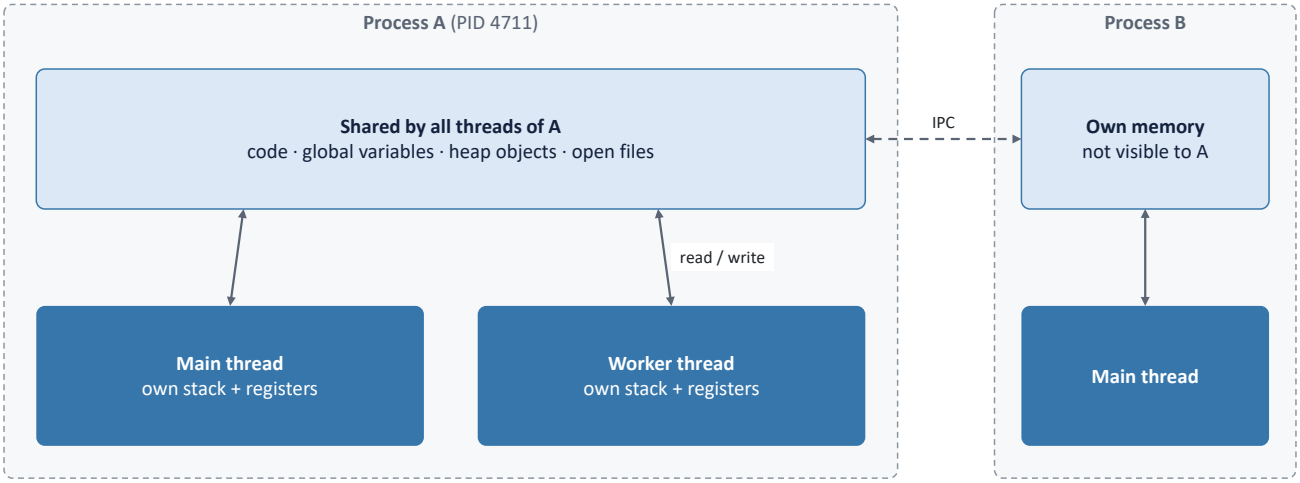


Key idea

Concurrency is about *structure*: dealing with many tasks at once. **Parallelism** is about *execution*: running many tasks at the same instant (Pike, 2012).

Processes and Threads

A **process** is a running program with its own memory. A **thread** is an independent flow of execution *inside* a process.



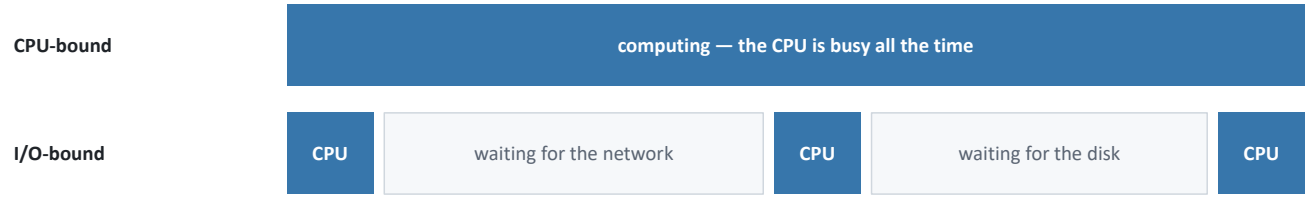
Threads vs Processes

	Threads	Processes
Memory	shared within the process	separate address spaces
Start-up and switching	cheap (microseconds)	expensive (new interpreter: ms)
Communication	shared objects — need locks	messages: pipes, queues (pickled)
Failure isolation	a segfault kills all threads	a crash stays in one process
CPU-bound Python (GIL build)	one core at a time	all cores in parallel
Python API	`threading`, `ThreadPoolExecutor`	`multiprocessing`, `ProcessPoolExecutor`

Key idea

Rule of thumb on the default CPython build: threads for *waiting*, processes for *computing*.

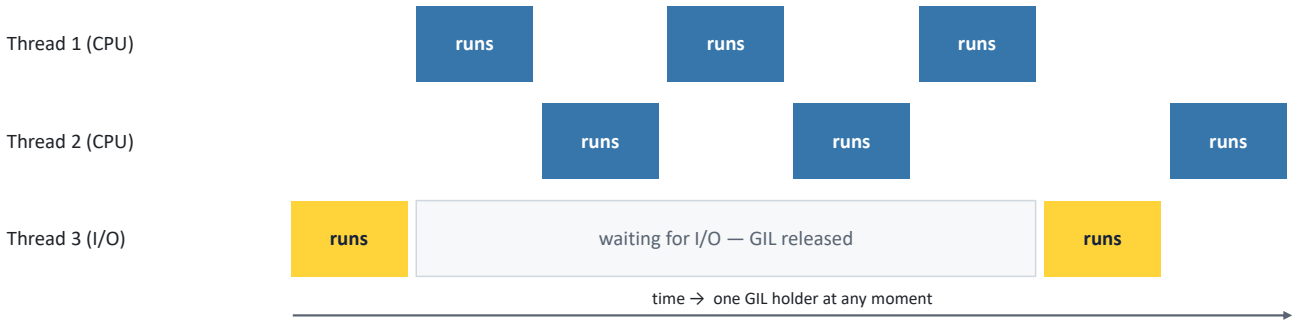
CPU-Bound vs I/O-Bound Work



	CPU-bound	I/O-bound
Limited by	processor speed	network, disk, database, user
Examples	image filters, parsing, simulations	web requests, DB queries, file copies
Threads on CPython (GIL)	no speed-up	large speed-up: GIL released while waiting
Typical tool	processes: `ProcessPoolExecutor`	threads or `asyncio`

The Global Interpreter Lock (GIL)

- The **GIL** is a lock inside CPython: a thread must hold it to run Python bytecode
- The holder switches every 5 ms (`sys.getswitchinterval()`) or when a thread blocks on I/O
- Why: reference counting and interpreter internals were not designed to be thread-safe



The GIL in Action: CPU-Bound Threads

```python
import time
from concurrent.futures import ThreadPoolExecutor


def count_down(n: int) -> None:
    while n > 0:                    # pure-Python CPU work
        n -= 1


if __name__ == "__main__":
    start = time.perf_counter()
    count_down(20_000_000)
    count_down(20_000_000)
    print(f"sequential: {time.perf_counter() - start:.2f} s")
    start = time.perf_counter()
    with ThreadPoolExecutor() as pool:   # each call in a thread
        pool.submit(count_down, 20_000_000)
        pool.submit(count_down, 20_000_000)
    print(f"2 threads:  {time.perf_counter() - start:.2f} s")
```

SAMPLE OUTPUT

```
sequential: 0.82 s
2 threads:  0.84 s
```

★ Two threads on many cores are *not* faster: only one can run Python bytecode at a time.

Working Around the GIL

Multiprocessing

- One interpreter, one GIL per process
- `ProcessPoolExecutor`, `multiprocessing`
- Best for CPU-bound pure Python

C Extensions

- NumPy, `hashlib`, `zlib` release the GIL
- Heavy loops run in parallel in C
- Also Cython, Numba (Module 11)

asyncio

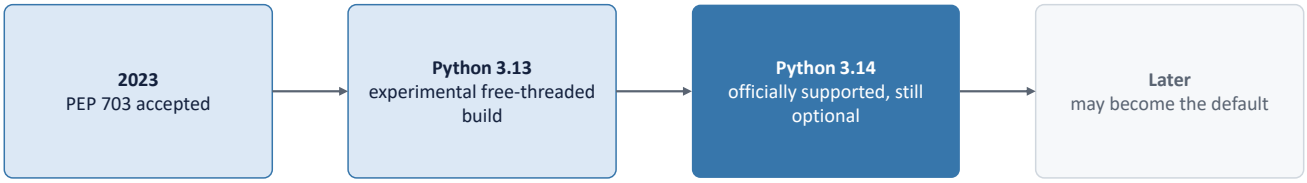
- Thousands of I/O tasks in one thread
- No threads, no GIL contention
- No help for CPU-bound code

✗ **Common misconception**

"asyncio gets around the GIL" — only for **I/O-bound** work. Coroutines share one thread and never run in parallel.

1 · CONCURRENCY CONCEPTS

The Future of the GIL



TERMINAL

```
$ python3.14t -c "import sys; print(sys._is_gil_enabled())"
False
$ python3.14 -c "import sys; print(sys._is_gil_enabled())"
True
```

Python version note

3.13: experimental free-threaded build (PEP 703). 3.14: officially supported but optional — not the default build (PEP 779).

QUIZ

Quick Check: The GIL

Which statement about CPython's GIL is true?

A. It makes all Python code thread-safe, so locks are unnecessary

B. Only one thread per process executes Python bytecode at any moment

C. It stops threads from speeding up programs that download many files

D. Python 3.14's default build no longer has a GIL

ANSWER

Quick Check: The GIL

Which statement about CPython's GIL is true?

A. It makes all Python code thread-safe, so locks are unnecessary

B. Only one thread per process executes Python bytecode at any moment

C. It stops threads from speeding up programs that download many files

D. Python 3.14's default build no longer has a GIL

Why

The GIL serialises bytecode execution inside one process. It is released while threads wait for I/O, so downloads still benefit; your own multi-step operations still need locks; and the free-threaded build is optional in 3.14.

02

Working with Threads

Creating, starting, joining and naming threads with the threading module

- Thread objects: target, args, start and join
- The thread lifecycle
- Names, daemon threads and subclassing Thread

Creating and Starting Threads

two_threads.py

```python
import threading
import time

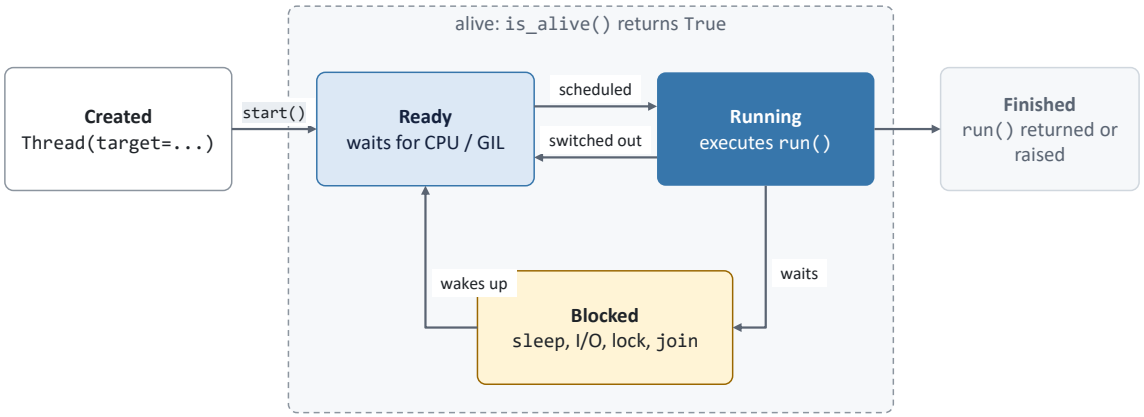

def report(name: str, delay: float) -> None:
    for step in range(1, 4):
        time.sleep(delay)
        print(f"{name}: step {step}")


if __name__ == "__main__":
    fast = threading.Thread(target=report, args=("fast", 0.1))
    slow = threading.Thread(target=report, args=("slow", 0.25))
    fast.start()          # returns immediately
    slow.start()
    fast.join()           # wait until fast has finished
    slow.join()
    print("main: both threads finished")
```

SAMPLE OUTPUT

```
fast: step 1
fast: step 2
slow: step 1
fast: step 3
slow: step 2
slow: step 3
main: both threads finished
```


The Thread Lifecycle



Watch out

`start()` may be called only once per `Thread` object; a finished thread cannot be restarted — create a new one.

Waiting for Threads with join()

lottery.py

```python
import random
import threading

tickets: list[list[int]] = []


def draw_numbers(highest: int, size: int) -> None:
    numbers = random.sample(range(1, highest + 1), size)
    tickets.append(sorted(numbers))


if __name__ == "__main__":
    threads = [threading.Thread(target=draw_numbers, args=(49, 6))
               for _ in range(100)]
    for thread in threads:          # start all first ...
        thread.start()
    for thread in threads:          # ... then wait for all
        thread.join()
    print(len(tickets), "tickets, first:", tickets[0])
```

SAMPLE OUTPUT

```
100 tickets, first: [3, 7, 19, 22, 30, 41]
```


Naming and Identifying Threads

```python
import threading
import time


def poll_sensor(interval: float) -> None:
    time.sleep(interval)
    name = threading.current_thread().name
    print(f"{name}: reading after {interval} s")


if __name__ == "__main__":
    sensors = [threading.Thread(target=poll_sensor, args=(i / 10,),
                                name=f"sensor-{i}") for i in (1, 2)]
    for sensor in sensors:
        sensor.start()
    print(threading.active_count(), "threads alive")
    for sensor in sensors:
        sensor.join()
```

SAMPLE OUTPUT

```
3 threads alive
sensor-1: reading after 0.1 s
sensor-2: reading after 0.2 s
```

Tip

In real programs use `logging` with `%(threadName)s` in the format: it is thread-safe and records which thread wrote each line.

Daemon Threads

- Non-daemon thread: the program waits for it at exit
- **Daemon** thread: killed abruptly when the main thread ends
- Set `daemon=True` *before* `start()`
- Use for background helpers: heartbeats, caches, monitors

> **Watch out**
> Daemon threads stop mid-work: `finally` blocks and file flushes may never run. For a clean shutdown use an `Event` (section 4).

daemon.py

```python
import threading
import time


def heartbeat() -> None:
    while True:           # never ends
        print("heartbeat")
        time.sleep(0.1)


if __name__ == "__main__":
    threading.Thread(target=heartbeat,
                     daemon=True).start()
    time.sleep(0.25)
    print("main: exiting")
```

SAMPLE OUTPUT

```
heartbeat
heartbeat
heartbeat
main: exiting
```


Subclassing Thread

downloader.py

```python
import threading
import time


class Downloader(threading.Thread):
    def __init__(self, url: str) -> None:
        super().__init__()           # always call the base initializer
        self.url = url
        self.size = 0

    def run(self) -> None:           # start() calls run() in the new thread
        time.sleep(0.1)              # stands in for network I/O
        self.size = len(self.url) * 100


if __name__ == "__main__":
    workers = [Downloader(url) for url in ("a.com", "python.org")]
    for worker in workers:
        worker.start()
    for worker in workers:
        worker.join()
        print(worker.url, worker.size)
```

OUTPUT

```
a.com 500
python.org 1000
```

TRACE THE CODE

Which Outputs Are Possible?

```python
import threading


def say(word: str) -> None:
    print(word + "1")
    print(word + "2")


if __name__ == "__main__":
    worker = threading.Thread(target=say,
                              args=("A",))
    worker.start()
    say("B")
    worker.join()
```

Each `print` writes one line; lines are shown left to right. Which can occur?

Output 1

`A1 A2 B1 B2`

Output 2

`B1 A1 B2 A2`

Output 3

`A2 A1 B1 B2`

Output 4

`B1 B2 A1 A2`

ANSWER

Which Outputs Are Possible?

Output	Possible?	Why
`A1 A2 B1 B2`	yes	the worker runs completely before `say("B")`
`B1 A1 B2 A2`	yes	the threads alternate after every line
`A2 A1 B1 B2`	**no**	one thread always runs its own lines in order
`B1 B2 A1 A2`	yes	the main thread runs first; the worker starts late

Key idea

Threads interleave unpredictably, but each thread keeps its **own program order**.

FIND THE BUG

Spot the Thread Bugs

The author wants three downloads to run *concurrently*. Find three bugs.

```python
import threading


def download(url: str) -> None:
    print("downloading", url)


if __name__ == "__main__":
    for url in ["a.com", "b.org"]:
        worker = threading.Thread(target=download(url))
        worker.start()
        worker.join()
    extra = threading.Thread(target=download, args=("c.net"))
    extra.start()
```

✓ Hint: for each download ask *which thread runs it* and *when it starts and finishes*.

03

Race Conditions and Locks

Lost updates, mutual exclusion, thread-safe classes and deadlock

- A race condition, step by step
- Lock, RLock and the with statement
- A thread-safe class
- Deadlock and lock ordering

A Race Condition in Action

```python
import threading
import time

counter = 0

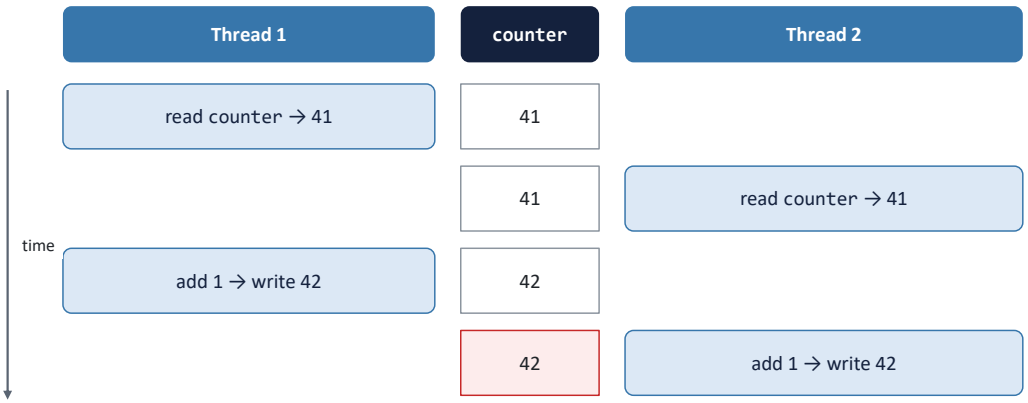

def increment() -> None:
    global counter
    for _ in range(10_000):
        value = counter           # 1. read
        time.sleep(0)             # invite a thread switch
        counter = value + 1       # 2. modify, 3. write


if __name__ == "__main__":
    threads = [threading.Thread(target=increment) for _ in range(4)]
    for thread in threads:
        thread.start()
    for thread in threads:
        thread.join()
    print(f"expected 40000, got {counter}")
```

SAMPLE OUTPUT

```
expected 40000, got 10313
```


A Lost Update, Step by Step



Not atomic

`counter += 1` compiles to several bytecodes — `LOAD_GLOBAL`, `BINARY_OP`, `STORE_GLOBAL`. A switch between them loses an update: two increments, result 42, not 43.

Mutual Exclusion with a Lock

- A **critical section** is code that must not run in two threads at once
- `threading.Lock` gives **mutual exclusion**: at most one holder at a time
- `acquire()` blocks while another thread holds the lock; `release()` frees it

explicit acquire / release

```
lock = threading.Lock()

lock.acquire()          # blocks while taken
try:
    counter += 1        # critical section
finally:
    lock.release()      # always runs
```

preferred: with lock

```
lock = threading.Lock()

with lock:              # acquire() on entry
    counter += 1        # critical section
# release() on exit, even after an exception
```

Key idea

Prefer `with lock:` — the lock is a context manager (Module 4), so it is released on every path out of the block.

Fixing the Counter with a Lock

```
import threading
import time

counter = 0
counter_lock = threading.Lock()


def increment() -> None:
    global counter
    for _ in range(10_000):
        with counter_lock:
            value = counter
            time.sleep(0)
            counter = value + 1


if __name__ == "__main__":
    threads = [threading.Thread(target=increment) for _ in range(4)]
    for thread in threads:
        thread.start()
    for thread in threads:
        thread.join()
    print(f"expected 40000, got {counter}")
```

OUTPUT

```
expected 40000, got 40000
```

Reentrant Locks: RLock

inventory.py

```python
import threading


class Inventory:
    def __init__(self) -> None:
        self.items: dict[str, int] = {}
        self._lock = threading.RLock()      # Lock() would hang below

    def add(self, item: str, qty: int) -> None:
        with self._lock:
            self.items[item] = self.items.get(item, 0) + qty

    def add_many(self, batch: dict[str, int]) -> None:
        with self._lock:                    # 1st acquire
            for item, qty in batch.items():
                self.add(item, qty)         # 2nd acquire, same thread


stock = Inventory()
stock.add_many({"apple": 3, "pear": 2})
print(stock.items)
```

OUTPUT

```
{'apple': 3, 'pear': 2}
```


A Thread-Safe Class

account.py

```python
import threading


class InsufficientBalance(Exception):
    """Raised when a withdrawal exceeds the balance."""


class Account:
    """A bank account that many threads may use at once."""

    def __init__(self, iban: str, balance: float = 0.0) -> None:
        self.iban = iban
        self.balance = balance
        self._lock = threading.Lock()       # one lock per object

    def deposit(self, amount: float) -> None:
        with self._lock:
            self.balance += amount

    def withdraw(self, amount: float) -> None:
        with self._lock:                    # check AND act together
            if amount > self.balance:
                raise InsufficientBalance(self.iban)
            self.balance -= amount
```

FIND THE BUG

Spot the Race in withdraw()

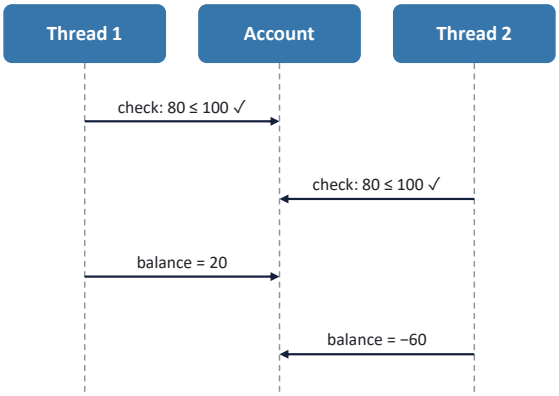
Two threads call `account.withdraw(80)` on an account holding 100. Sometimes both succeed and the balance becomes −60. Explain how, then fix the method.

```python
def withdraw(self, amount: float) -> None:
    if amount > self.balance:
        raise InsufficientBalance(self.iban)
    with self._lock:
        self.balance -= amount
```

✓ Hint: draw a timeline with one column per thread and write down the balance after every step.

ANSWER

Spot the Race in withdraw()




```python
def withdraw(self, amount: float) -> None:
    with self._lock:      # check + act
        if amount > self.balance:
            raise InsufficientBalance(self.iban)
        self.balance -= amount
```

★ **Key idea**

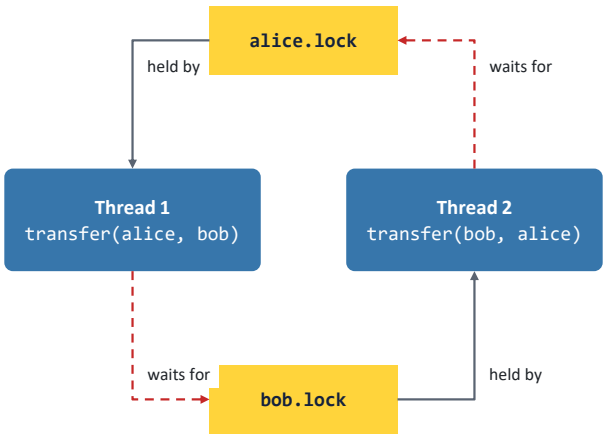
Check-then-act must be one atomic step: put the test and the update under the same lock.

Deadlock: Two Threads, Two Locks

deadlock.py (hangs)

```python
def transfer(source, target, amount):
    with source.lock:
        time.sleep(0.01)    # widen window
        with target.lock:   # waits forever
            source.balance -= amount
            target.balance += amount


# thread 1: transfer(alice, bob, 10)
# thread 2: transfer(bob, alice, 20)
```



Key idea

Deadlock: every thread waits for a lock held by another. No exception, no CPU use — the program just hangs.

Preventing Deadlock

- **Lock ordering**: all threads acquire locks in one global order
- Hold as few locks as possible, for as short as possible
- Use `acquire(timeout=...)` to detect trouble and back off
- Never call unknown code (callbacks) while holding a lock
- Prefer queues and executors to hand-made locking

lock ordering

```python
def transfer(source, target, amount):
    first, second = sorted(
        (source, target),
        key=lambda acc: acc.number)
    with first.lock, second.lock:
        source.balance -= amount
        target.balance += amount
```

Tip

Both threads now lock the account with the lower number first, so a cycle cannot form.

QUIZ

Quick Check: Deadlock

`worker_1` takes `lock_a` then `lock_b`; `worker_2` takes `lock_b` then `lock_a`. Which change guarantees they can never deadlock?

- **A** Replace both locks with `threading.RLock()`
- **B** Add `time.sleep(0.1)` between the two `with` statements
- **C** Make both workers take `lock_a` first, then `lock_b`
- **D** Start both threads with `daemon=True`

ANSWER

Quick Check: Deadlock

`worker_1` takes `lock_a` then `lock_b`; `worker_2` takes `lock_b` then `lock_a`. Which change guarantees they can never deadlock?

- **A** Replace both locks with `threading.RLock()`
- **B** Add `time.sleep(0.1)` between the two `with` statements
- **C** Make both workers take `lock_a` first, then `lock_b` ✓
- **D** Start both threads with `daemon=True`

★ **Why**

A common acquisition order removes the circular wait. An `RLock` only lets the *same* thread re-acquire; a sleep changes timing, not possibility; daemon threads are merely killed at exit.

04

Coordination Primitives

Semaphores, events, conditions, barriers and queues

- Limit concurrency with semaphores
- Signal with events and conditions
- Meet at a barrier
- Producer–consumer with queues

4 · COORDINATION PRIMITIVES

Beyond Locks: The Coordination Toolbox

Primitive	Coordinates	Typical use
`Lock`, `RLock`	exclusive access	protect shared state
`Semaphore(n)`	at most *n* holders	limit connections or downloads
`Event`	a one-way flag	"start now", "stop now"
`Condition`	waiting until a state is true	custom buffers, state changes
`Barrier(n)`	*n* threads meet at a point	phase-by-phase algorithms
`queue.Queue`	passing data between threads	producer–consumer pipelines

Tip

Reach for `queue.Queue` first: it hides the locks and conditions behind `put()` and `get()`.

Semaphores: Limiting Concurrent Access

- A **semaphore** holds a counter of *permits*
- `acquire()` takes a permit — it blocks while the counter is 0
- `release()` returns a permit and wakes one waiting thread
- `BoundedSemaphore` raises `ValueError` if released too often: prefer it



Semaphore Example: Three Downloads at a Time

downloads.py

```python
import threading
import time

slots = threading.BoundedSemaphore(3)    # 3 permits
START = time.perf_counter()


def download(item: int) -> None:
    with slots:                          # acquire ... release
        elapsed = time.perf_counter() - START
        print(f"{elapsed:.1f} s  start item {item}")
        time.sleep(0.2)                  # simulated network I/O


if __name__ == "__main__":
    threads = [threading.Thread(target=download, args=(item,))
               for item in range(1, 7)]
    for thread in threads:
        thread.start()
    for thread in threads:
        thread.join()
```

SAMPLE OUTPUT

```
0.0 s  start item 1
0.0 s  start item 2
0.0 s  start item 3
0.2 s  start item 4
0.2 s  start item 5
0.2 s  start item 6
```

Signalling with Event

An `Event` is a thread-safe flag: `set()`, `clear()`, `is_set()` and `wait(timeout)`.

```python
import threading
import time

stop = threading.Event()


def monitor(interval: float) -> None:
    checks = 0
    while not stop.is_set():
        checks += 1
        stop.wait(interval)          # sleeps, but wakes early on set()
    print(f"monitor: stopped after {checks} checks")


if __name__ == "__main__":
    worker = threading.Thread(target=monitor, args=(0.1,))
    worker.start()
    time.sleep(0.35)
    stop.set()                       # ask the worker to finish
    worker.join()
    print("main: done")
```

SAMPLE OUTPUT

```
monitor: stopped after 4 checks
main: done
```


Waiting for a State with Condition

A `Condition` combines a lock with *wait until notified*; `wait_for(predicate)` re-checks the state for you.

```python
import threading

orders: list[str] = []
changed = threading.Condition()


def kitchen() -> None:
    with changed:
        changed.wait_for(lambda: len(orders) >= 2)  # releases lock
        print("kitchen: cooking", orders)

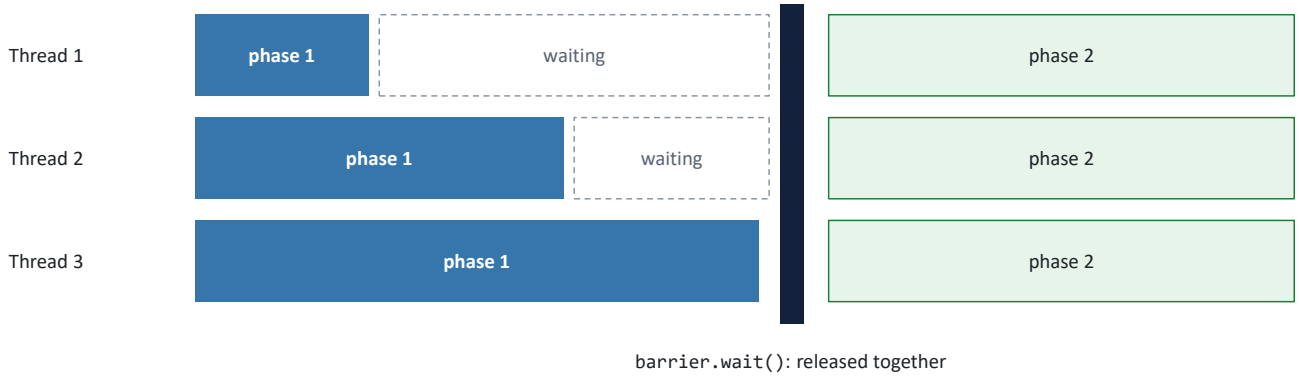

if __name__ == "__main__":
    cook = threading.Thread(target=kitchen)
    cook.start()
    for order in ("soup", "salad"):
        with changed:
            orders.append(order)
            changed.notify_all()     # "the state has changed"
    cook.join()
```

OUTPUT

```
kitchen: cooking ['soup', 'salad']
```

Synchronizing Phases with a Barrier

- A **barrier** blocks threads until a fixed number of them has arrived
- Each thread calls `barrier.wait()`; the last arrival releases everyone
- Reusable: after release the barrier is ready for the next phase



Barrier Example: Load, Then Process

barrier.py

```python
import threading
import time

barrier = threading.Barrier(3)

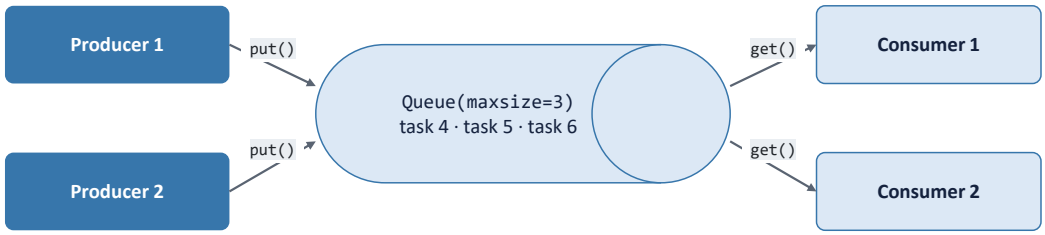

def worker(name: str, load_time: float) -> None:
    time.sleep(load_time)                # phase 1: uneven work
    print(f"{name}: data loaded, waiting")
    barrier.wait()                       # block until all 3 arrive
    print(f"{name}: processing")         # phase 2 starts together


if __name__ == "__main__":
    threads = [threading.Thread(target=worker, args=(f"w{i}", i / 10))
               for i in range(1, 4)]
    for thread in threads:
        thread.start()
    for thread in threads:
        thread.join()
```

SAMPLE OUTPUT

```
w1: data loaded, waiting
w2: data loaded, waiting
w3: data loaded, waiting
w3: processing
w1: processing
w2: processing
```

Producer–Consumer with queue.Queue



- `put()` blocks while the queue is full — **backpressure** on fast producers
- `get()` blocks while it is empty — idle consumers use no CPU
- `task_done()` + `join()` tell when every item has been processed

Python version note

Python 3.13+: `Queue.shutdown()` makes further `put()` calls fail and wakes blocked consumers with `queue.ShutDown`.

Producer–Consumer: The Code

pipeline.py

```python
import queue
import threading

tasks: queue.Queue[int] = queue.Queue(maxsize=3)


def consumer(name: str) -> None:
    while True:
        task = tasks.get()                # blocks while empty
        print(f"{name} processed task {task}")
        tasks.task_done()                 # mark this task finished


if __name__ == "__main__":
    for name in ("c1", "c2"):
        threading.Thread(target=consumer, args=(name,),
                         daemon=True).start()
    for task in range(1, 6):              # the producer
        tasks.put(task)                   # blocks while full
    tasks.join()                          # waits for 5 task_done()
    print("all tasks processed")
```

SAMPLE OUTPUT

```
c1 processed task 1
c1 processed task 2
c1 processed task 3
c2 processed task 4
c2 processed task 5
all tasks processed
```

QUICK CHECK

Choose the Primitive

#	Scenario
1	At most 4 threads may talk to the licence server at the same time
2	All worker threads must stop when the user clicks *Cancel*
3	Log lines from 10 threads must be written by one writer thread
4	3 simulation threads must finish step *k* before any starts step *k* + 1
5	Several threads update a shared `dict` used as a cache
6	A thread must sleep until a buffer holds at least 10 items

ANSWER

Choose the Primitive

#	Primitive	Why
1	`BoundedSemaphore(4)`	limits concurrent holders to four
2	`Event`	one `set()` is seen by every worker's `is_set()` / `wait()`
3	`queue.Queue`	many producers, one consumer; no shared file handle
4	`Barrier(3)`	nobody passes until all three arrive
5	`Lock`	protects read–modify–write of shared state
6	`Condition` (or a `Queue`)	`wait_for(lambda: len(buffer) >= 10)`

05

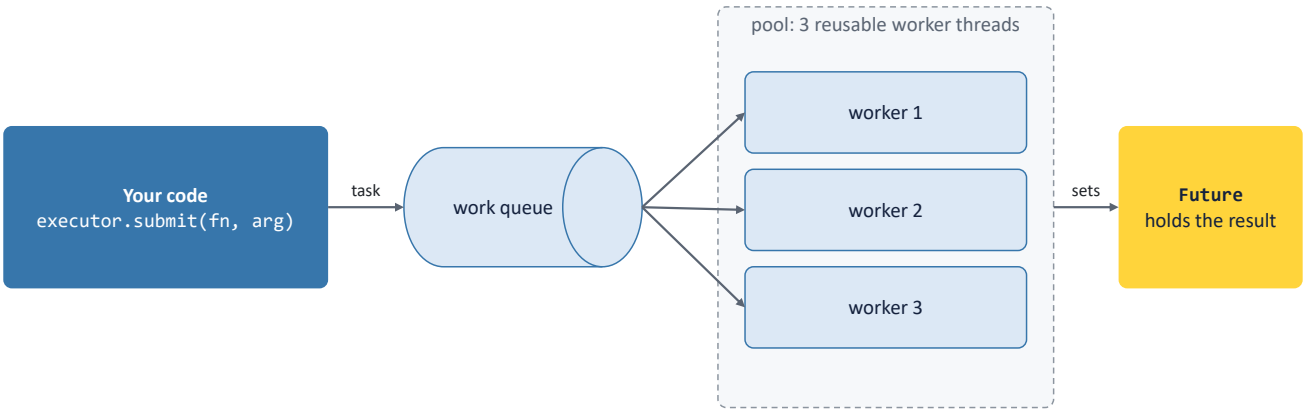
Executors and Processes

Thread pools, futures, multiprocessing and choosing a model

- Thread pools: submit, map and as_completed
- Futures and exceptions
- Process pools, multiprocessing and the main-module guard
- Choosing a concurrency model

Thread Pools: Reuse Instead of Re-Create

- Creating an OS thread per task is costly; a **thread pool** keeps workers alive
- `concurrent.futures`: one API for thread pools *and* process pools
- `submit()` returns a **future** at once: a placeholder for the result



Submitting Work with submit()

lottery_pool.py

```python
import random
from concurrent.futures import ThreadPoolExecutor


def draw_numbers(highest: int, size: int) -> list[int]:
    return sorted(random.sample(range(1, highest + 1), size))


if __name__ == "__main__":
    with ThreadPoolExecutor(max_workers=4) as executor:
        futures = [executor.submit(draw_numbers, 49, 6)
                   for _ in range(100)]
    tickets = [future.result() for future in futures]
    print(len(tickets), "tickets, first:", tickets[0])
```

SAMPLE OUTPUT

```
100 tickets, first: [4, 11, 17, 25, 38, 46]
```

Key idea

`submit()` returns immediately; `result()` blocks until that task is done — and re-raises its exception.

Collecting Results: map vs as_completed

```python
import time
from concurrent.futures import ThreadPoolExecutor, as_completed


def fetch(delay: float) -> str:
    time.sleep(delay)                       # simulated download
    return f"page-{delay}"


if __name__ == "__main__":
    delays = [0.3, 0.1, 0.2]
    with ThreadPoolExecutor() as executor:
        print(list(executor.map(fetch, delays)))  # input order
        futures = [executor.submit(fetch, d) for d in delays]
        for future in as_completed(futures):      # finish order
            print(future.result())
```

SAMPLE OUTPUT

```
['page-0.3', 'page-0.1', 'page-0.2']
page-0.1
page-0.2
page-0.3
```

	executor.map(fn, items)	as_completed(futures)
Order	same as the input	as tasks finish
Use when	you need results in order	you want to react to each result early

Futures: Results and Exceptions

```
>>> from concurrent.futures import ThreadPoolExecutor
>>> with ThreadPoolExecutor() as executor:
...     good = executor.submit(int, "42")
...     bad = executor.submit(int, "n/a")
...
>>> good.result()
42
>>> bad.done()
True
>>> bad.result()
Traceback (most recent call last):
  ...
ValueError: invalid literal for int() with base 10: 'n/a'
```

Method	Purpose
`result(timeout)`	wait; return value or re-raise
`exception()`	wait; return the exception or None
`done()`	finished, failed or cancelled?
`cancel()`	cancel if not started yet
`add_done_callback(fn)`	call `fn(future)` when done

Processes with multiprocessing

```python
import time
from multiprocessing import Process


def count_down(n: int) -> None:
    while n > 0:
        n -= 1


if __name__ == "__main__":              # required: see next slide
    start = time.perf_counter()
    workers = [Process(target=count_down, args=(20_000_000,))
               for _ in range(2)]
    for worker in workers:
        worker.start()
    for worker in workers:
        worker.join()
    print(f"2 processes: {time.perf_counter() - start:.2f} s")
```

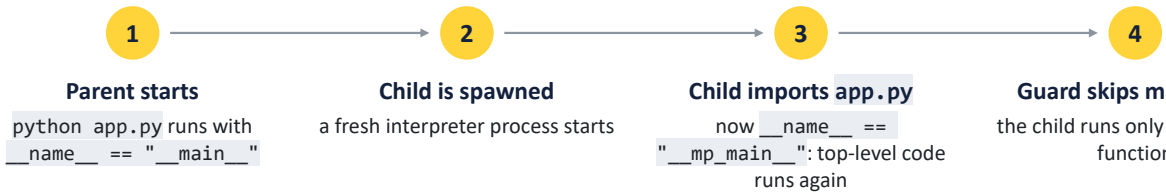
SAMPLE OUTPUT

```
2 processes: 0.51 s
```



Own interpreter and own GIL per process: both loops run in parallel (threads: 0.84 s).

Why the __main__ Guard Is Required



Watch out

Without the guard, step 3 would start new processes again — Python stops this with a `RuntimeError`. Keep process-starting code under `if __name__ == "__main__":`.

Python version note

Default start methods: `spawn` on Windows and macOS; Linux moved from `fork` to `forkserver` in Python 3.14. Guarded code works with all of them.

ProcessPoolExecutor for CPU-Bound Work

```python
import math
import time
from concurrent.futures import ProcessPoolExecutor


def is_prime(n: int) -> bool:
    return all(n % d for d in range(2, math.isqrt(n) + 1))


def count_primes(limit: int) -> int:
    return sum(1 for n in range(2, limit) if is_prime(n))


if __name__ == "__main__":
    limits = [80_000] * 4
    start = time.perf_counter()
    with ProcessPoolExecutor() as executor:
        counts = list(executor.map(count_primes, limits))
    print(counts, f"{time.perf_counter() - start:.2f} s")
```

SAMPLE OUTPUT

```
[7837, 7837, 7837, 7837] 0.29 s
```

- Same `Executor` API as threads: just swap the class name
- Arguments and results are **pickled**: top-level functions, picklable data

5 · EXECUTORS AND PROCESSES

Choosing a Concurrency Model

Workload	Use	Why
A few blocking I/O calls: files, HTTP, database drivers	`ThreadPoolExecutor`	simple; the GIL is released while waiting
Thousands of network connections	`asyncio`	one thread, cheap tasks; needs async libraries
CPU-bound pure-Python code	`ProcessPoolExecutor`	one GIL per process: all cores
CPU-bound work in NumPy or other C extensions	threads	the extension releases the GIL
A blocking call inside async code	`asyncio.to_thread`	keeps the event loop responsive

Python version note

On the free-threaded build (3.14t) threads can also run CPU-bound Python in parallel — once all your dependencies support it.

TRACE THE CODE

Does the Child Share the Counter?

```python
from multiprocessing import Process

counter = 0


def increment() -> None:
    global counter
    counter += 1


if __name__ == "__main__":
    child = Process(target=increment)
    child.start()
    child.join()
    print(counter)
```

What does the parent process print?

A
`0`

B
`1`

C
`2`

D
an exception

Follow-up

Would the answer change with `threading.Thread` instead of `Process`?

ANSWER

Does the Child Share the Counter?

OUTPUT

```
0
```

Key idea

A. The child increments its *own* copy of `counter`. Processes do not share memory.

To get data back	Use
a return value	`ProcessPoolExecutor` future
a stream of items	`multiprocessing.Queue`
one shared number	`multiprocessing.Value`
shared dict / list	`multiprocessing.Manager()`

fix: return the value

```python
from concurrent.futures import ProcessPoolExecutor


def increment(value: int) -> int:
    return value + 1


if __name__ == "__main__":
    with ProcessPoolExecutor() as pool:
        future = pool.submit(increment, 0)
    print(future.result())
```

OUTPUT

```
1
```


TRY IT

Speed Up the Link Checker

The loop below checks 10 links one after another (≈ 2 s). Use a `ThreadPoolExecutor` so it finishes in ≈ 0.2 s and keeps the results in link order.

```python
import time


def check(url: str) -> int:
    time.sleep(0.2)                  # simulated HTTP request
    return 200


urls = [f"https://example.com/page{i}" for i in range(10)]
statuses = [check(url) for url in urls]
print(statuses)
```

Tip

Hint: which executor method keeps input order?

ANSWER

Speed Up the Link Checker

```python
import time
from concurrent.futures import ThreadPoolExecutor


def check(url: str) -> int:
    time.sleep(0.2)                  # simulated HTTP request
    return 200


if __name__ == "__main__":
    urls = [f"https://example.com/page{i}" for i in range(10)]
    start = time.perf_counter()
    with ThreadPoolExecutor(max_workers=10) as executor:
        statuses = list(executor.map(check, urls))
    print(statuses)
    print(f"{time.perf_counter() - start:.1f} s")
```

SAMPLE OUTPUT

```
[200, 200, 200, 200, 200, 200, 200, 200, 200, 200]
0.2 s
```


06

Asynchronous Programming with asyncio

Coroutines, the event loop, tasks, TaskGroup and timeouts

- Blocking vs non-blocking waiting
- Coroutines, async/await and the event loop
- Tasks, gather and TaskGroup
- Never block the loop; timeouts

Blocking Code: One Thing at a Time

sync_count.py

```python
import time


def count() -> None:
    print("One")
    time.sleep(1)
    print("Two")


def main() -> None:
    for _ in range(3):
        count()


if __name__ == "__main__":
    start = time.perf_counter()
    main()
    elapsed = time.perf_counter() - start
    print(f"executed in {elapsed:0.2f} seconds.")
```

SAMPLE OUTPUT

```
One
Two
One
Two
One
Two
executed in 3.00 seconds.
```


Blocking vs Non-Blocking Waiting



Key idea

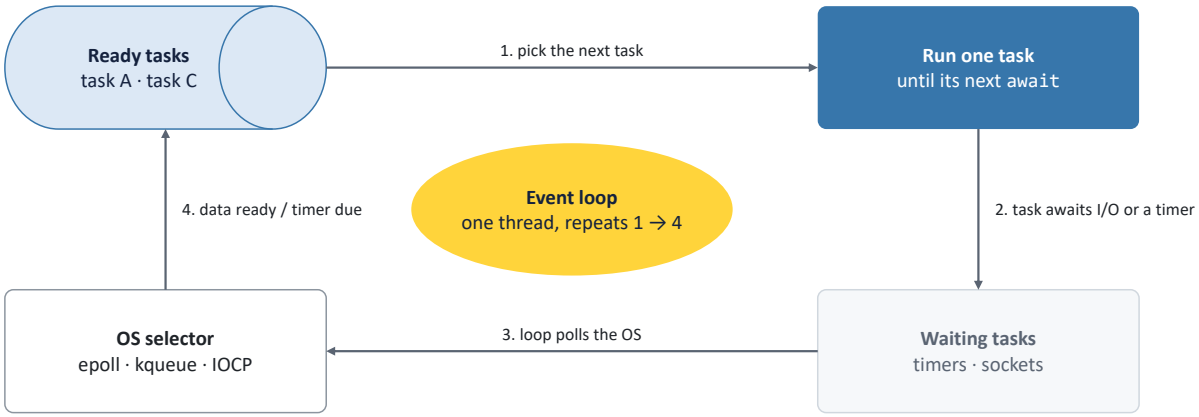
While one coroutine waits, the event loop runs the others: **concurrency in one thread**, switching only at `await`.

Coroutines: async def and await

- `async def` defines a **coroutine function**
- Calling it creates a **coroutine** object — nothing runs yet
- `await` runs an **awaitable** and suspends the caller until it is done
- `asyncio.run()` starts an **event loop** and runs the top-level coroutine

```
>>> import asyncio
>>> async def greet(name):
...     await asyncio.sleep(0.1)
...     return f"Hello, {name}!"
...
>>> coro = greet("Ada")
>>> coro
<coroutine object greet at 0x...>
>>> asyncio.run(coro)
'Hello, Ada!'
```


The Event Loop



Watch out

One task runs at a time; it keeps the CPU until its next `await`. A task that never awaits blocks every other task.

Running Coroutines Concurrently with gather

async_count.py

```python
import asyncio
import time


async def count() -> None:
    print("One")
    await asyncio.sleep(1)
    print("Two")


async def main() -> None:
    await asyncio.gather(count(), count(), count())


if __name__ == "__main__":
    start = time.perf_counter()
    asyncio.run(main())
    elapsed = time.perf_counter() - start
    print(f"executed in {elapsed:0.2f} seconds.")
```

SAMPLE OUTPUT

```
One
One
One
Two
Two
Two
executed in 1.00 seconds.
```


Tasks: Start Now, Await Later

```python
import asyncio
import time


async def fetch(name: str) -> str:
    await asyncio.sleep(0.3)              # simulated network call
    return name


async def main() -> None:
    start = time.perf_counter()
    await fetch("users")                  # 0.3 s ...
    await fetch("orders")                 # ... + 0.3 s
    print(f"one after another: {time.perf_counter() - start:.1f} s")
    start = time.perf_counter()
    users = asyncio.create_task(fetch("users"))    # both run
    orders = asyncio.create_task(fetch("orders"))  # from now on
    print(await users, await orders)
    print(f"as tasks:          {time.perf_counter() - start:.1f} s")


if __name__ == "__main__":
    asyncio.run(main())
```

SAMPLE OUTPUT

```
one after another: 0.6 s
users orders
as tasks:          0.3 s
```

Structured Concurrency with TaskGroup

```python
import asyncio

TEMPERATURES = {"Istanbul": 21.5, "Ankara": 17.0, "Izmir": 24.0}


async def fetch_temperature(city: str) -> float:
    await asyncio.sleep(0.1)                # simulated API call
    return TEMPERATURES[city]


async def main() -> None:
    async with asyncio.TaskGroup() as group:
        tasks = {city: group.create_task(fetch_temperature(city))
                 for city in TEMPERATURES}
    for city, task in tasks.items():        # all tasks are done here
        print(f"{city}: {task.result()} °C")


if __name__ == "__main__":
    asyncio.run(main())
```

OUTPUT

```
Istanbul: 21.5 °C
Ankara: 17.0 °C
Izmir: 24.0 °C
```

No task outlives the `async with` block; if one task fails, the others are cancelled.

When a Task Fails: except*

```python
import asyncio


async def fetch(city: str, delay: float) -> str:
    await asyncio.sleep(delay)
    if city == "Atlantis":
        raise LookupError(f"unknown city: {city}")
    return city


async def main() -> None:
    try:
        async with asyncio.TaskGroup() as group:
            slow = group.create_task(fetch("Ankara", 1.0))
            group.create_task(fetch("Atlantis", 0.1))
    except* LookupError as failures:
        for error in failures.exceptions:
            print("failed:", error)
    print("slow task cancelled:", slow.cancelled())


if __name__ == "__main__":
    asyncio.run(main())
```

OUTPUT

```
failed: unknown city: Atlantis
slow task cancelled: True
```

Never Block the Event Loop

```python
import asyncio
import time


async def heartbeat() -> None:
    start = time.perf_counter()
    for _ in range(4):
        print(f"tick at {time.perf_counter() - start:.1f} s")
        await asyncio.sleep(0.1)


async def blocking_job() -> None:
    time.sleep(0.3)            # blocks the whole event loop!


async def main() -> None:
    await asyncio.gather(heartbeat(), blocking_job())


if __name__ == "__main__":
    asyncio.run(main())
```

SAMPLE OUTPUT

```
tick at 0.0 s
tick at 0.3 s
tick at 0.4 s
tick at 0.5 s
```


Offloading Blocking Calls with to_thread

```python
import asyncio
import time


async def heartbeat() -> None:
    start = time.perf_counter()
    for _ in range(4):
        print(f"tick at {time.perf_counter() - start:.1f} s")
        await asyncio.sleep(0.1)


def legacy_download() -> str:
    time.sleep(0.3)                # a blocking library call
    return "report.csv"


async def main() -> None:
    _, name = await asyncio.gather(
        heartbeat(), asyncio.to_thread(legacy_download))
    print("downloaded", name)


if __name__ == "__main__":
    asyncio.run(main())
```

SAMPLE OUTPUT

```
tick at 0.0 s
tick at 0.1 s
tick at 0.2 s
tick at 0.3 s
downloaded report.csv
```

Timeouts with asyncio.timeout

timeout.py

```python
import asyncio


async def slow_query() -> str:
    await asyncio.sleep(2)                 # e.g. an overloaded server
    return "rows"


async def main() -> None:
    try:
        async with asyncio.timeout(0.5):   # deadline for the block
            print(await slow_query())
    except TimeoutError:
        print("query cancelled after 0.5 s")


if __name__ == "__main__":
    asyncio.run(main())
```

OUTPUT

```
query cancelled after 0.5 s
```

Python version note

`asyncio.timeout()` is new in Python 3.11; for a single awaitable `asyncio.wait_for(aw, seconds)` does the same.

TRACE THE CODE

Predict the Order of Events

```python
import asyncio


async def worker(name: str, delay: float) -> None:
    print(f"{name} start")
    await asyncio.sleep(delay)
    print(f"{name} end")


async def main() -> None:
    task = asyncio.create_task(worker("A", 0.2))
    print("main continues")
    await worker("B", 0.1)
    await task


if __name__ == "__main__":
    asyncio.run(main())
```

ANSWER

Predict the Order of Events

OUTPUT

```
main continues
B start
A start
B end
A end
```

- `create_task` only *schedules* A; it runs when `main` first awaits
- `await worker("B", 0.1)` runs B directly until its `sleep`: `B start`
- Now the loop runs A until its `sleep`: `A start`
- B's 0.1 s timer expires first, then A's 0.2 s timer

QUIZ

Quick Check: asyncio

Which statement about asyncio is true?

A. Coroutines in one event loop run in parallel on several cores

B. A coroutine can be paused only at an `await`

C. `time.sleep(1)` in a coroutine lets other tasks run meanwhile

D. Calling an `async def` function runs its body immediately

ANSWER

Quick Check: asyncio

Which statement about asyncio is true?

A. Coroutines in one event loop run in parallel on several cores

B. A coroutine can be paused only at an `await`

C. `time.sleep(1)` in a coroutine lets other tasks run meanwhile

D. Calling an `async def` function runs its body immediately

Why

asyncio is cooperative: the loop switches tasks only where a coroutine awaits. It runs in one thread, `time.sleep` blocks the loop, and calling a coroutine function only creates a coroutine object.

MODULE 05 · PRACTICE

Practice Problems

1. Parallel Word Count

- Count words in 8 large text files
- Use a thread pool, then a process pool
- Time both and explain the difference

2. Bounded Buffer

- Write `BoundedBuffer` with `put()` / `get()`
- Block when full or empty, using a `Condition`
- Test with two producers, two consumers

3. Dining Philosophers

- Five threads share five locks (forks)
- Reproduce the deadlock with `sleep`
- Fix it with lock ordering

4. Async Link Checker

- Check 20 simulated URLs with `TaskGroup`
- At most 5 at a time: `asyncio.Semaphore`
- Each check within 1 s: `asyncio.timeout`

MODULE 05 · SUMMARY

Key Takeaways

- ✓ Concurrency structures many tasks; parallelism runs them at the same instant
- ✓ With the GIL, threads speed up I/O-bound work; use processes for CPU-bound Python
- ✓ Protect shared state with `with lock:`; take several locks in one global order
- ✓ Prefer high-level tools: `queue.Queue` and `concurrent.futures` executors
- ✓ asyncio: one thread, many tasks, switches only at `await` — never block the loop
- ✓ Use `TaskGroup` and `asyncio.timeout` for structured, time-limited async code

NEXT UP Module 06 — Introduction to Software Architectures